



ETCD High Availability & Backup



What is etcd?

- etcd = Kubernetes brain 🧠
- etcd is the core database of Kubernetes that stores:
 - 📦 Pods, Services, Deployments
 - 🔑 Secrets & ConfigMaps
 - 🔑 RBAC rules
 - 🌐 Cluster state
 - 👉 Without etcd → Kubernetes stops working 🚨



Simple Analogy

- etcd = 🧠 Brain
- Kubernetes = 🏃 Body 👉 If brain stops → body stops



How Does etcd Stay Available?

- 👉 To ensure high availability ,etcd runs as a cluster of multiple nodes .



ETCD High Availability Concepts

◆ 1. Cluster with Multiple Members

- Example:
- Instead of one server:
 - Node1 (Leader)
 - Node2 (Follower)
 - Node3 (Follower)
 - If one node fails → cluster still works (others continue)

◆ 2. Raft Consensus Algorithm

- etcd uses Raft Consensus Algorithm to manage consistency and automatic leader election

- Only one node (leader) makes changes at a time.
- Followers replicate data, All etcd members store the same data.
- Leader failure → new leader elected automatically 🔄 (No manual action required)
- 👉 Self-healing system

♦ 3. Quorum-Based Writes

- etcd requires quorum (majority) to accept any write.
- Always use an odd number of nodes to form a quorum. (Always use **odd number of nodes (3,5,7))

📖 Formula:

$$\text{Quorum} = (\text{Total Nodes} / 2) + 1$$

Example:

- In a 3-node etcd cluster → quorum = 2
- 5 nodes → quorum = 3
- 👉 Minimum majority must be alive
- So, even if 1 node goes down , the cluster still works. At least 2 must be healthy to maintain write availability .

♦ 4. Auto Failover

- 🔄 What Happens if Leader Fails?
 - If Leader crashes 💥
 - Followers detect failure
 - Raft holds an election 🗳️
 - A new leader is chosen from the followers 👑
 - ✓ Cluster continues working, 👉 No manual intervention needed
 - 👉 Ensures High Availability 🚀

🧠 In Short:

- etcd ensures High Availability ✓ through clustering, Raft-based consensus 🔄 , quorum writes 📊 , and automatic leader election .

 Feature	 What It Means	 How It Works	 Why It Matters
 Clustering	Multiple etcd nodes	👉 Runs as a distributed cluster (typically 3 or 5 nodes)	🚫 Avoids single point of failure
 Raft Consensus	Agreement protocol	👉 Ensures All nodes agree on data before commit	🔒 Strong consistency
 Quorum Writes	Majority approval	👉 Write succeeds only if majority ($N/2 + 1$) nodes agree	🛡️ Prevents data corruption
 Leader Election	One leader node	👉 Leader handles writes; followers replicate	⚙️ Ensures Coordinated updates

🔥 What if ALL etcd nodes fail?

- Kubernetes control plane stops ❌
 - 💥 No scheduling
 - 💥 No API access
- 👉 Only solution:
 - Restore snapshot
 - OR rebuild cluster

📁 Where is etcd data stored?

In kubeadm clusters:

`/var/lib/etcd`

🔒 etcd Certificates (Security)

- 👉 etcd is secured using certificates
- Think:
 - cert = ID
 - key = password

🤔 Why Certificates?

- etcd is secure by default. 👉 It asks: Are you authorized to access me? 🔒

Simple Analogy

etcd = Secure room 🏠

cert = ID card 🪪

key = Secret key 🔑

etcd TLS Certificates in kubeadm

 Purpose	 File	 What It Does	 Why It Matters
 CA (Trust)	/etc/kubernetes/pki/etcd/ca.crt	 Certificate Authority that signs all etcd certs	Establishes trust between components
 Certificate (ID)	/etc/kubernetes/pki/etcd/server.crt	 Identifies etcd server securely	Used for TLS authentication
 Private Key	/etc/kubernetes/pki/etcd/server.key	 Secret key paired with certificate	Enables encrypted communication

Verify Files

```
ls /etc/kubernetes/pki/etcd/
```

- Should show: ca.crt , server.crt and server.key

Take etcd Backup

```
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-snapshot.db \  
--endpoints=https://127.0.0.1:2379 \  
--cacert=/etc/kubernetes/pki/etcd/ca.crt \  

```

```
--cert=/etc/kubernetes/pki/etcd/server.crt \
--key=/etc/kubernetes/pki/etcd/server.key
```

- ✓ Secure connection
- ✓ Full cluster backup
- ✓ Saves .db file

Restore Command

```
ETCDCTL_API=3 etcdctl snapshot restore /backup/etcd-snapshot.db \
--data-dir /var/lib/etcd-restore
```

ETCD in EKS (Managed Kubernetes)

- ⚠ Important Limitation
 - ❌ No direct access to etcd in EKS
 - ❌ You Cannot Access: etcd server . Certificates and Data directory
 - ✅ AWS Handles: etcd backups , High availability and Recovery

Best Practices

 Practice	 What to Do	 Why It Matters	 Pro Tip
 Use odd number of nodes	Deploy 3, 5, or 7 nodes	 Ensures proper quorum ($N/2 + 1$)	★ 3-node cluster is most common
 Take regular backups	Schedule etcd snapshots	 Protect cluster state from data loss	 Automate with cron jobs
 Test restore process	Perform periodic restore drills	 Verify backups actually work	 Test in staging environment
 Secure certificates	Protect .crt and .key files	 Prevent unauthorized access	 Restrict file permissions
 Monitor etcd health	Track metrics & endpoints	 Detect issues early	 Use Prometheus alerts

- In EKS, etcd is fully managed by AWS , so we don't handle etcd backups manually.

kubeadm vs EKS (etcd Management)

 Feature	 kubeadm (Self-managed)	 EKS (Managed)	 Explanation	 Impact
 etcd Access	 Full access	 No access	You can SSH and interact with etcd in kubeadm; hidden in EKS	Less control in EKS
 Backup	 Manual (<code>etcdctl snapshot</code>)	 AWS managed	You must schedule backups vs AWS handles control plane backups	Simpler in EKS
 Cert Files	 Available (<code>/etc/kubernetes/pki</code>)	 Not accessible	Full PKI control vs AWS-managed certificates	Security handled by AWS
 Responsibility	 User (you manage everything)	 AWS manages control plane	Self-managed vs managed service	Trade-off: control vs convenience

Common Mistakes

-  Running single-node etcd
-  Not taking backups
-  Ignoring quorum rules
-  Losing cert files

Final Summary

 Concept	 Meaning
 etcd	Kubernetes brain (stores cluster state)
 Type	Distributed key-value store

 Concept	 Meaning
 High Availability	Achieved via clustering + Raft consensus
 Quorum	Majority ($N/2 + 1$) ensures consistency
 Backup	Critical for disaster recovery
 Certificates	Stored at <code>/etc/kubernetes/pki/etcd/</code>
 EKS	AWS manages etcd (no direct access)

Ultimate One-Line

- etcd ensures Kubernetes reliability using clustering, Raft consensus, and quorum-based operations, while backups protect against total failure.